

[← Back to tasks](#)Python Programming I... > **Week 2** > Task 1

Week 2 · Task 1

In Progress

Guide included

File Handling & Working with External Data

Beginner

1-2 Days

Python Development

ABOUT THIS TASK

In this task, interns will learn how to work with files in Python. They will read data from files, write output to files, and understand how Python handles external data storage. This task introduces real-world programming scenarios where data persistence is required.

Task in Brief

6 parts

Everything you need to complete this task — follow each section in order.

YOUR PATH

1 Overview



2 Steps



3 GitHub Repo



4 File Structure



5 Practice



6 Submit

1

What You Will Do

Read this first to understand the goal of this task.

In this task, you will master the fundamentals of file handling in Python by reading from and writing to external text files. You will learn how to persist data across program executions and understand the differences between various file modes like read, write, and append. By the end of this task, you will deliver functional Python scripts and a brief conceptual document demonstrating your understanding of data persistence.

2

Step-by-Step Instructions

Complete each step in order before submitting your work.

- 1 Create a project directory and set up a local Git repository to track your progress.
- 2 Write a Python script that reads data from an input text file, processes the text (e.g., counts words or lines), and handles potential file-not-found errors using try-except blocks.
- 3 Write a second Python script that takes user input or structured data and writes/appends it to a new output text file.
- 4 Create a short markdown document explaining the differences between 'r', 'w', and 'a' file modes, including when to use each.
- 5 Run both scripts, capture screenshots of the terminal outputs and the generated files, and save them in an assets folder.
- 6 Commit your code, documentation, and screenshots to your local Git repository and push them to a public GitHub repository.

3

What Your GitHub Repo Must Include

Make sure your repository has everything listed below.

- ✓ A well-structured README.md file explaining the project, how to run the scripts, and a summary of what you learned.
- ✓ Clean, well-commented Python source code (.py files) organized in a dedicated directory.

- ✔ A docs folder containing the file modes explanation document in markdown format.
- ✔ An assets folder containing clear image evidence of the scripts running successfully and the resulting file outputs.
- ✔ A clean commit history showing incremental progress with descriptive commit messages.

4**Suggested Project Structure**

Organize your files like this in your repository.

- 📄 README.md
- 📄 src/read_file.py
- 📄 src/write_file.py
- 📄 docs/file_modes.md
- 📄 data/input.txt
- 📄 assets/screenshots/

5**Hands-On Practice**

Complete these mini-tasks and add the required files to your repo.

1 Read and Analyze Text File**Create:** `src/read_file.py`

Create a script that reads an input text file ('input.txt'), counts the total number of words, and prints the result to the console. Implement error handling to manage missing files gracefully.

2 Write and Append User Data**Create:** `src/write_file.py`

Create a script that prompts the user to enter three of their favorite quotes, then writes these quotes to a new file ('quotes.txt'), ensuring each quote is on

a new line. Run it multiple times using append mode to add more quotes without overwriting.

6

Ready to Submit?

Use the submission form below when you have completed all steps.

Submit the link to your public GitHub repository containing all the required code, documentation, and screenshots. Reviewers will verify code functionality, error handling, repository structure, and the clarity of your file modes explanation.

Learning Resources

Reference courses to help you complete this task

PROGRAM-WIDE

[Introduction To Python Programming](#)